- 딥러닝
- = 손실 함수를 최소화 하는 거임

- **Gradient Descent**가 쓰임
- 최적인 세타를 찾아야 함.
  기울기 방향으로 이동하며 → 최소값을 찾음
  - 전역 최소를 보장하지는 않음.

$$\theta^{k+1} \leftarrow \theta^k - \eta \nabla_\theta J(\theta^k)$$

- → one iteration.
- eta → learning rate 역할을 하고, 한번에 얼마나 변할지를 정함.

Stochastic Gradient Descent(SGD)

- 데이터가 너무 많아서, 학습하기 힘들 때가 많음
- sub set에만 GD를 하는 것과 전체 데이터에 하는 것과 비슷하다고 가정함.

If batchsize=1 then θ is updated after each example.

If batchsize=N (full set) then this is standard gradient descent.

- 이렇게 하면 빠름. + 로컬 minimum에서 벗어날 수 있음
  좀 불안정함.

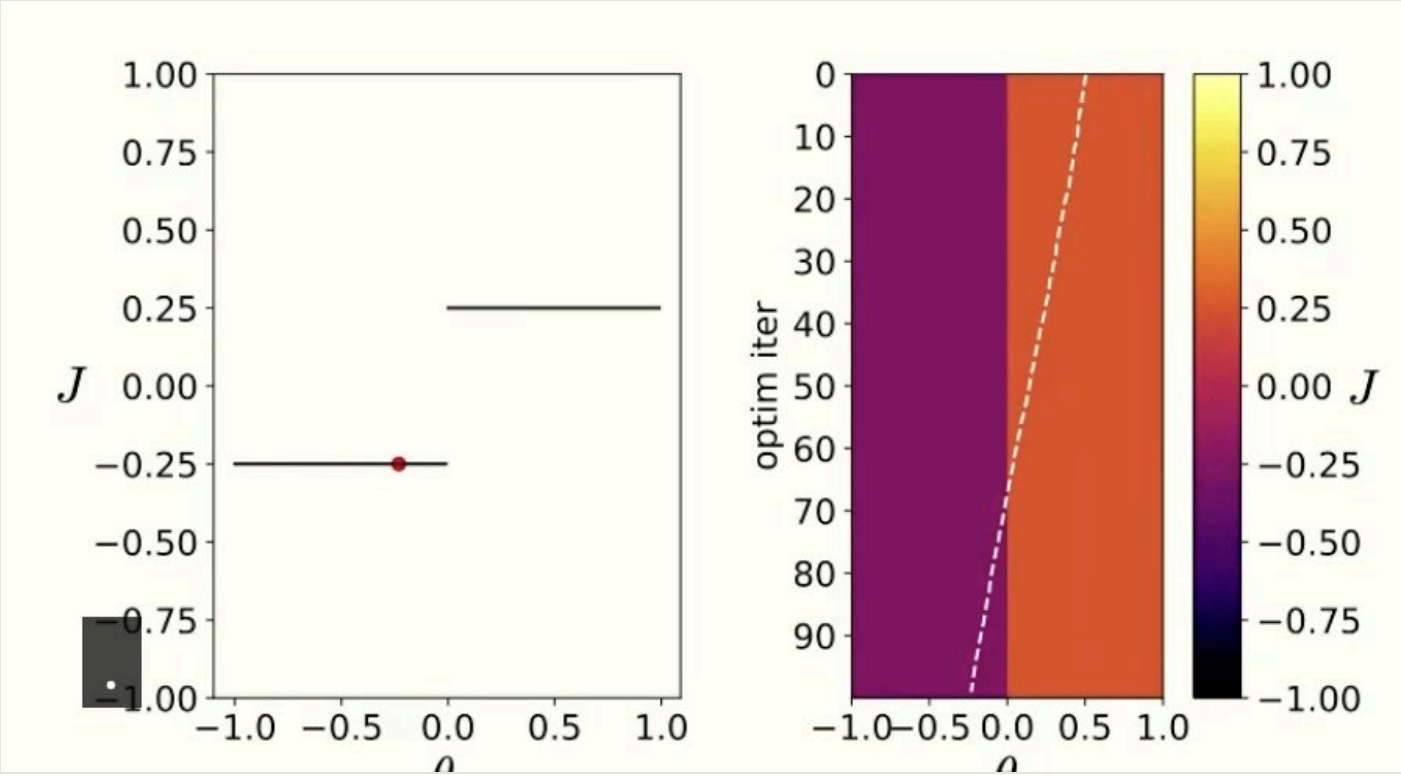
Momentum

- GD에서 업데이트 방향으로 **관성**을 가지고 업데이트 하게 됨

$$\theta^{t+1} \leftarrow \theta^t - \eta \nabla f(\theta^t) - \alpha m^t$$

- 너무 작은 모멘텀 → 최소값에 적당히 천천히 감
  적당한 모멘텀 → 최소값에 더 빨리 도달 할 수 있음
  너무 높은 모멘텀 → 최소값을 지나가버림

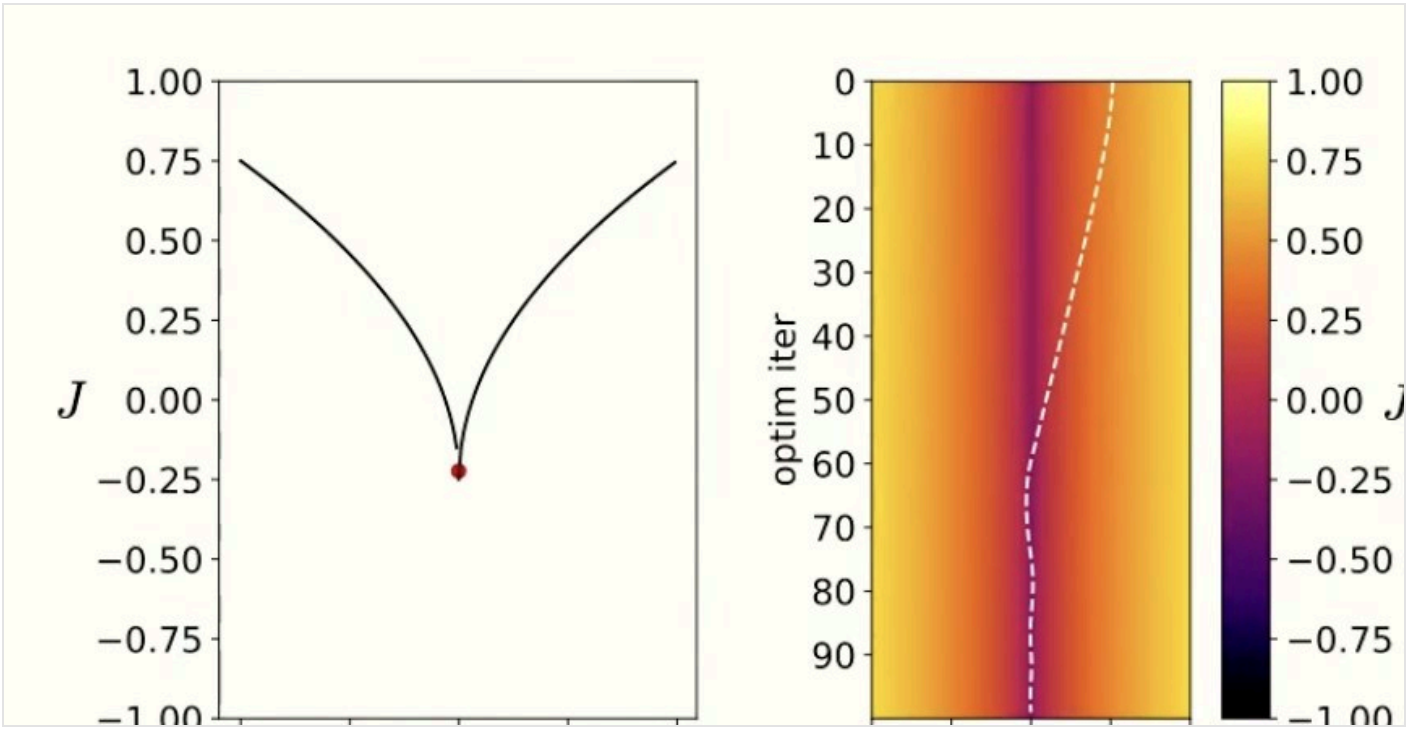
Evolution Strategies



$$\epsilon_i \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$
$$s_i = J(\theta + \sigma \epsilon_i)$$

$$\theta^{k+1} \leftarrow \theta^k - \eta \frac{1}{\sigma M} \sum_{i=1}^{M} s_i \epsilon_i$$

Gradient clipping



- 기울기가 너무 커지면 → 다시 스케일링하면 됨.

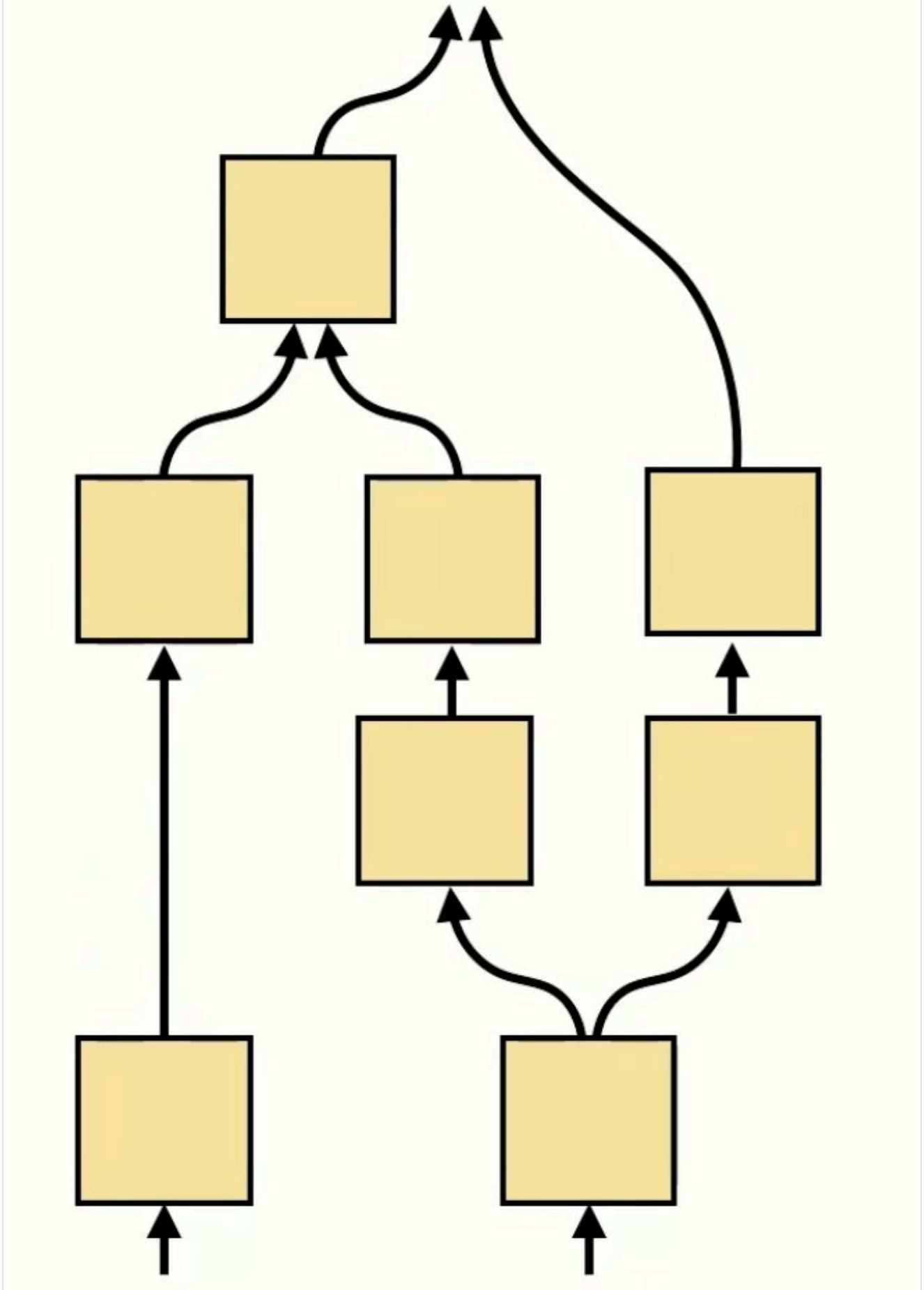
$$\mathbf{v} = \nabla_\theta J(\theta^k)$$
$$\theta^{k+1} \leftarrow \theta^k - \eta[\texttt{clip}(v_1, -m, m), \ldots, \texttt{clip}(v_M, -m, m)]^\mathsf{T}$$

- 매개변수의 변화량을 계산 → 그 방향으로 이동(여기까지 같음)
  but 최대값이 제한됨.

Loss function

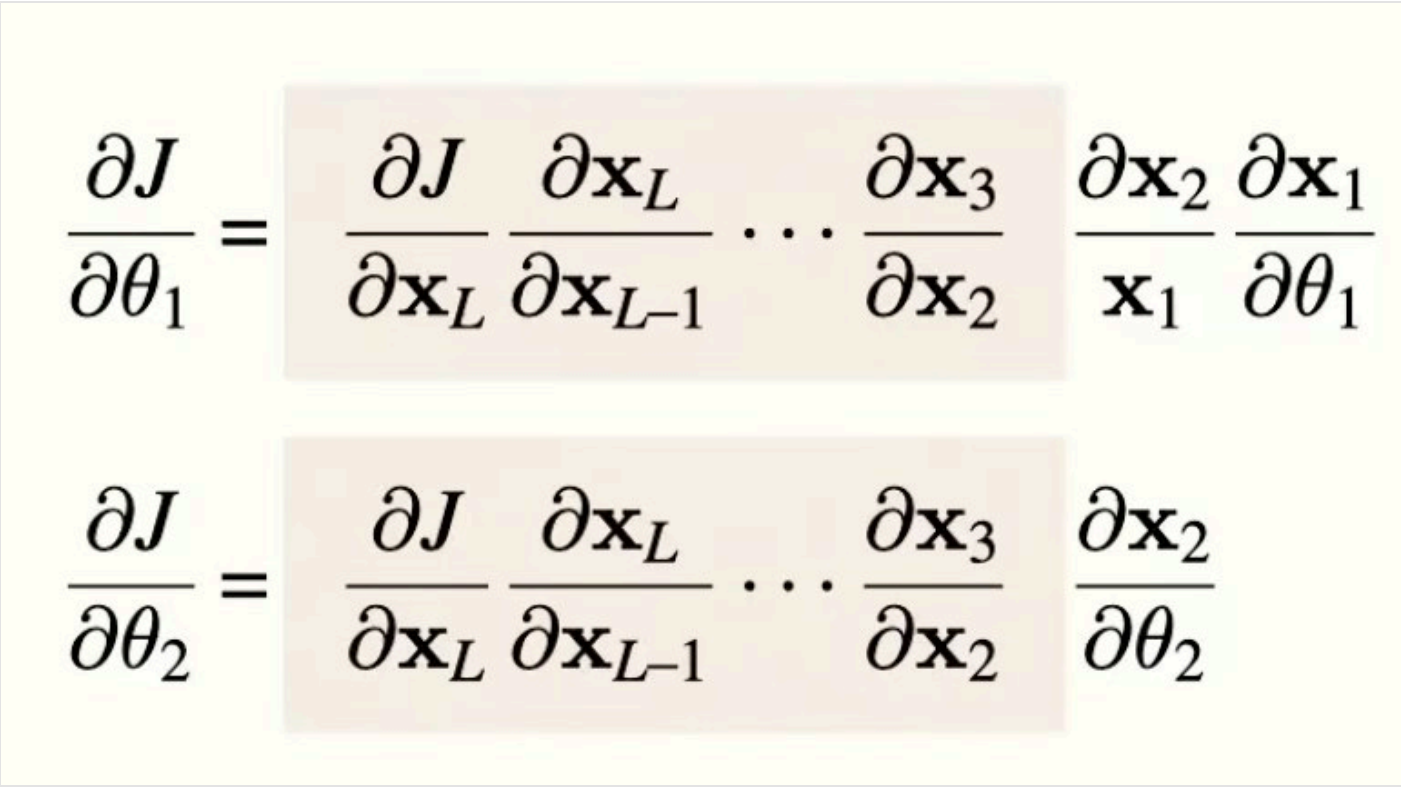
- 연속 / 미분 가능 / 매끄러워야 함.
- EX> ReLU = max(0,z)
  smooth하지 않음.
  → GeLU같은 걸 씀.(smooth함)

Computation Graphs



- 아래가 input, 위로 갈수록 output

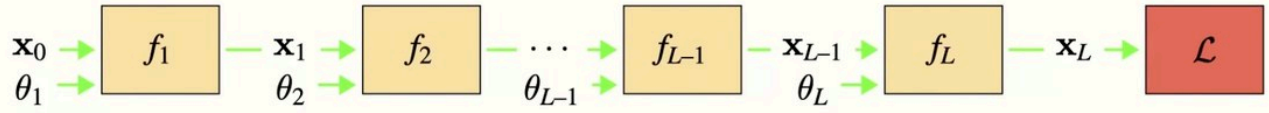
Backpropagation



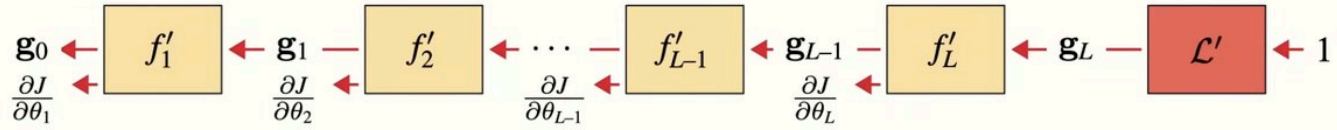
- Computation Graphs에서 세타에 대한 미분을 계산하기 위한 방법.
  Chain rule을 이용해 backward 방향으로 미분값을 얻음

The Full Algorithm: Forward, Then Backward

Forward:



Backward:



Update:

$$\theta^{i+1} \leftarrow \theta^i - \eta \left(\frac{\partial J}{\partial \theta} \right)^\mathsf{T}$$

… and repeat

- forward → backward → update 반복하면 training

- Linear layer
- Forward = 그냥 f = Wx 곱하면 됨.
- Backprop

$$\mathbf{g}_{\text{in}} = \mathbf{g}_{\text{out}} \cdot \frac{\partial f(\mathbf{x}_{\text{in}}, \mathbf{W})}{\partial \mathbf{x}_{\text{in}}} = \mathbf{g}_{\text{out}} \cdot \frac{\partial \mathbf{x}_{\text{out}}}{\partial \mathbf{x}_{\text{in}}}$$

$$\frac{\partial \mathbf{x}_{\text{out}_i}}{\partial \mathbf{x}_{\text{in}_j}} = \mathbf{W}_{ij} \longrightarrow \frac{\partial f(\mathbf{x}_{\text{in}}, \mathbf{W})}{\partial \mathbf{x}_{\text{in}}} = \mathbf{W}$$

- therefore

$$\mathbf{g}_{\text{in}} = \mathbf{g}_{\text{out}} \cdot \mathbf{W}$$

- loss function을 구하려면

$$\frac{\partial J}{\partial \mathbf{W}} = \mathbf{g}_{\text{out}} \cdot \frac{\partial f(\mathbf{x}_{\text{in}}, \mathbf{W})}{\partial \mathbf{W}} = \mathbf{g}_{\text{out}} \cdot \frac{\partial \mathbf{x}_{\text{out}}}{\partial \mathbf{W}}$$

- gout → chainrule 역할

$$\frac{\partial J}{\partial \mathbf{W}_{ij}} = \frac{\partial J}{\partial \mathbf{x}_{\text{out}_i}} \cdot \frac{\partial \mathbf{x}_{\text{out}_i}}{\partial \mathbf{W}_{ij}} = \frac{\partial J}{\partial \mathbf{x}_{\text{out}_i}} \cdot \mathbf{x}_{\text{in}_j}$$

$$\frac{\partial \mathbf{x}_{\text{out}_i}}{\partial \mathbf{W}_{ij}} = \mathbf{x}_{\text{in}_j}$$

$$\frac{\partial J}{\partial \mathbf{W}} = \mathbf{x}_{\text{in}} \cdot \frac{\partial J}{\partial \mathbf{x}_{\text{out}}} = \mathbf{x}_{\text{in}} \cdot \mathbf{g}_{\text{out}}$$

- update 할 때 위 식을 쓰게 됨.

Differentiable programming

- Easy to optimize
  compositional “block based programming”